

Optimal Resizable Arrays 论文评述

张艺博 徐黄浩

2026 年 7 月 22 日

摘要

本文评述 Robert E. Tarjan 与 Uri Zwick 的论文 *Optimal resizable arrays*。论文讨论的问题很直接：一个数组既要支持 $O(1)$ 随机访问，又要在末尾不断增长和收缩，能否比普通的几何扩容更省空间？作者把平时占用的额外空间和扩缩容时短暂使用的空间分开计算，并据此给出一族可以调节的分层结构。本文主要介绍这一思路是怎样产生的、数据结构如何实现，以及论文所说的“最优”具体成立在什么范围内。

目录

1	引言与论文定位	4
1.1	研究背景与核心问题	4
1.2	论文定位与评述视角	5
2	问题定义与理论模型	5
2.1	模型定义与复杂度目标	5
2.1.1	可变长数组的抽象接口	6
2.1.2	内存块模型	6
2.1.3	两类空间开销	7
2.1.4	论文的复杂度目标	7
3	文献调研与相关工作	8
3.1	摊还分析与传统方法	8
3.2	传统工作	8
3.3	理论脉络与下界背景	9
3.3.1	Brodnik 等人的空间下界	9
3.3.2	Tarjan-Zwick 对下界视角的扩展	9
4	论文核心方法	10
4.1	整体构造与技术路线	10
4.1.1	整体构造与技术路线：从空间下界到构造目标	10
4.2	空间优化机制	12
4.2.1	$r = 3$ 的两级块结构	12
4.2.2	一般 r 的分层块结构	13
4.2.3	访问时间与均摊更新时间	14
4.2.4	去除稳定空间中的 r 因子	14
5	正确性证明与理论分析	15
5.1	上界分析	15
5.1.1	上界分析：一般 r 构造的均摊代价	15
5.1.2	访问与修改操作的最坏 $O(1)$ 时间	18
5.2	下界分析	21
5.2.1	标准实现及下界的适用范围	21
5.2.2	(N, k, ℓ) -growth game	22
5.2.3	二项式阈值与游戏下界	23

5.2.4	从真实结构到 $\Omega(r)$ 下界	23
6	技术直觉与方法评价	25
6.1	技术直觉与示例解释	25
6.1.1	示例解释	26
6.2	创新点、局限性与后续影响	26
6.2.1	核心创新与理论贡献	26
6.2.2	模型假设与定理边界	26
6.2.3	均摊更新与工程可行性	27
6.2.4	开放问题与后续意义	28
7	总结	28

1 引言与论文定位

1.1 研究背景与核心问题

数组和可变数组是广泛应用的数据结构，可变数组的内存效率一直是研究者的研究热点。本文所研究的可变数组指的是，可以在末尾添加或者删除元素的数组[1]。为此，很自然想到，如何解决可变长数组的空间分配问题？一个典型场景是：内存已分配的空间已经被填满，此时需要插入新元素，却不能直接在原有内存块末尾追加，因为紧邻的内存可能已被其他对象占用。因此，必须重新申请一块更大的内存，将原有元素全部复制过去，再释放旧块。

也就是：如何维护一个可以增长和收缩的数组，使它既支持按下标 $O(1)$ 访问，又尽可能节省空间，同时 grow/shrink 操作仍然高效？

此问题在工程中与实际中用处广泛，比如：Java ArrayList，C++ vector。但是，作者认为，之前的设计（满了扩容、空了减小）过于基础，尽管它是最常用的数据结构之一，但是它的空间最优性没有完全解决。

这一问题可以从**机制**与**策略**两个层面加以分离与理解。

机制层面

1. 系统提供固定大小的存储块（block）；
2. 存储块可以存放实际数据，也可以存放指向其他块的指针；
3. Grow 操作：申请新块，必要时进行数据迁移；
4. Shrink 操作：删除末尾元素，释放不再需要的空块。

策略层面

1. 块的大小如何选取？
2. 使用多少层指针？Grow 过程中块的大小如何变化？
3. Shrink 的触发条件是什么？何时进行全局重建（rebuild）？

将机制与策略解耦，有助于清晰地分析和设计可变长数组的数据结构。

经典方法：翻倍策略（Doubling） 工业界广泛采用的成熟做法是几何增长策略，其核心规则为：当数组满时，分配一块大小为当前数组两倍的连续内存，并将所有元素复制过去。这一策略的摊还代价为 $O(1)$ ，随机访问同样为 $O(1)$ 最坏情况时间[2]。标准倍增算法可以达到 $O(N)$ （更确切地说，是 $N + O(N)$ ）的空间开销和 $O(1)$ 的时间开

销，但其最坏时刻的空间浪费可达 50%。进一步，我们可以将这些策略总结成如下规律（后面第 3 章的传统方法部分会详细说明）：

假设每次数组满后，增长因子为 $(1 + \alpha)$ ，那么新数组刚扩展完毕时浪费率为 $\frac{\alpha}{1+\alpha}$ 。但是， α 也不能无限小：它越小，扩容越频繁，平均复制次数也越多。假设数组容量达到 C 后触发 grow，新容量变为 $C' = (1 + \alpha)C$ ，则每个操作的均摊成本为 $1/\alpha$ 。

本文的目标 能否将空间占用压缩至 $N + o(N)$ ，同时仍然保持 $O(1)$ 的随机访问时间？这正是 Tarjan 和 Zwick 论文试图回答的核心问题[1]。

核心思想 作者认为应该区分两种空间，第一种是存储空间：数组当前大小为 N ，平时为了存储它并支持访问，需要多少空间？第二种是 resize 临时空间：grow/shrink 的过程中，短暂需要多少额外空间？

1.2 论文定位与评述视角

这篇论文最值得注意的地方，不只是让空间复杂度更小，而是重新去研究：这些空间是在什么时候被用掉的？例如，几何扩容会一直留着一部分空位；分块数组平时要保存块指针；真正扩容时，又可能短暂同时保留新旧两个块。这几种开销虽然都算“额外空间”，性质却不一样。前两种会长期存在，最后一种只在某次操作中出现。作者因此用 $s(N)$ 表示平时的额外空间，用 $t(N)$ 表示扩缩容时的临时空间。

为什么要特意区分这两种空间？假设程序同时维护 K 个长度约为 N 的数组。每个数组平时多占的空间会达到 $K \cdot s(N)$ ，但通常不会有 K 个数组在同一时刻一起扩容。因此，我们可以接受某一次扩容临时多用一些空间，换取所有数组平时占用更少。论文中的参数 r 就是在控制这个交换： r 越大，平时留下的空位越少，但扩缩容需要的临时空间和搬移次数也会增加。

因此，接下来的内容围绕三个问题展开。第一，这种交换能不能由一个具体的数据结构做到？第二，省下空间以后，访问和扩缩容是否还足够快？第三，作者给出的结果是不是真的不能再改进？前两个问题对应论文的分层块结构，第三个问题对应空间乘积下界和 growth game。需要提前说明的是，论文的一些结论依赖 word-RAM 模型、均摊分析或标准实现，不能脱离这些条件来理解。

2 问题定义与理论模型

2.1 模型定义与复杂度目标

前面已经说明了论文想解决的问题，本节将写出准确的模型。这里研究的并不是可以在任意位置插入、删除的动态序列，而是更接近栈式数组：元素只能从末尾加入或删除

去，但访问操作仍要能够按下标任意访问。下面先列出数组支持的操作，再说明论文怎样计算空间开销[1]。

2.1.1 可变长数组的抽象接口

设当前数组为 A ，长度为 N 。论文中的可变长数组支持以下基本操作：

1. **Array()**：创建并返回一个初始为空的数组；
2. **Length()**：返回数组当前长度；
3. **Get(i)**：返回下标为 i 的元素，其中 $0 \leq i < N$ ；
4. **Set(i, a)**：将下标为 i 的元素修改为 a ，其中 $0 \leq i < N$ ；
5. **Grow(a)**：在数组末尾加入一个新元素 a ，使数组长度从 N 变为 $N + 1$ ；
6. **Shrink()**：删除数组末尾元素，使数组长度从 N 变为 $N - 1$ 。

这里最重要的限制是，插入和删除只发生在末端。若在中间插入一个元素，后面所有元素的下标都可能变化，那就变成了另一个更复杂的动态序列问题。为了本文后面的分层方法能够成立，这里只考虑数组在末尾变化。

2.1.2 内存块模型

论文假设底层内存管理系统允许申请和释放任意固定长度的数组块。一个可变长数组的具体实现由若干动态分配的块组成，主要包括三类：

1. **数据块 (data block)**：存放数组中的实际元素；
2. **索引块 (index block)**：存放指向数据块或其他索引块的指针；
3. **辅助块 (auxiliary block)**：存放块数量、块长度、层级计数器等辅助信息。

简单来说，普通动态数组通常只有一个连续内存块，满了以后只能申请更大的块并复制全部元素。论文允许把数组拆开放在多个块中，再用索引块找到它们。这样做多了一层结构，却也避免了始终为一个连续大数组预留很多空位。后面的分层结构就是建立在这一点上。

论文在该抽象中把一个元素或一个指针都视为占用一个机器字，并假设内存管理器可以申请和释放任意长度的固定数组块。为简化时间分析，单次申请或释放被视为 $O(1)$ ；即使长度为 L 的块需要 $O(L)$ 时间完成分配或释放，文中的均摊时间结论仍然成立。除一个作为入口的主索引块外，其余每个块都必须由某个索引块中的指针指向，否则算法无法访问该块。数据结构的空间占用则定义为当前所有已分配块的长度之和，而不是只统计实际存放元素的位置[1]。

2.1.3 两类空间开销

论文的关键建模创新是区分两类空间：

1. **存储与访问空间：** 当数组处于稳定状态、长度为 N 时，为了存储所有元素并支持 `Get` 和 `Set` 所需的空间；
2. **扩缩容临时空间：** 执行 `Grow` 或 `Shrink` 时，在元素移动、块合并或块拆分过程中短暂额外占用的空间。

用论文中的记号表示，设 $s(N)$ 和 $t(N)$ 是两个非递减函数。若一个实现满足以下条件，就称其为 $(s(N), t(N))$ 实现[1]：

$$\text{稳定存储空间} \leq N + s(N),$$

并且在扩缩容操作过程中最多使用：

$$\text{临时峰值空间} \leq N + t(N).$$

也就是说， $s(N)$ 记录数组平时比 N 个元素多占多少空间，其中包括指针、空位和辅助信息； $t(N)$ 则记录一次 `Grow` 或 `Shrink` 进行到一半时，最多会占多少空间。两者都把原来的 N 个元素算在总量里。作者把它们分开，是因为平时的冗余会一直存在，而扩缩容的峰值只持续很短时间。图 1 给出了两种空间的区别。

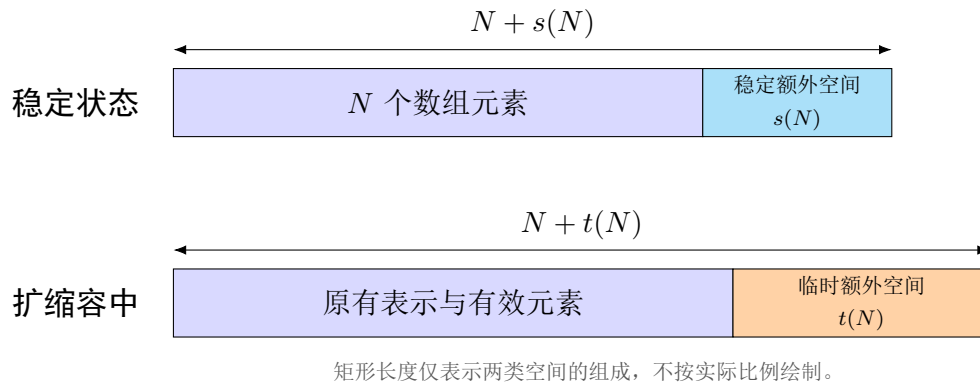


图 1: 稳定空间与扩缩容临时空间的区分（根据文献 [1] 整理）。

2.1.4 论文的复杂度目标

在上述模型下，论文希望同时达到以下目标：

1. `Get` 和 `Set` 操作保持 $O(1)$ 最坏情况时间；

2. Grow 和 Shrink 操作保持较低的均摊时间；
3. 稳定存储空间尽量接近信息论意义上的下限 N ；
4. 在允许一定临时空间的前提下，突破传统 $N + O(\sqrt{N})$ 空间界限。

最后的结果可以先记成一句话：对整数 $r \geq 2$ ，平时使用 $N + O(N^{1/r})$ 空间，扩缩容过程中最多使用 $N + O(N^{1-1/r})$ 空间；随机访问仍是 $O(1)$ ，扩缩容的均摊时间是 $O(r)$ 。后面出现的分层、计数器和下界证明，都是在解释这组结果为什么能够做到，以及为什么已经接近最好。

3 文献调研与相关工作

3.1 摊还分析与传统方法

在进行相关工作阐述之前，我们先解释一下摊还分析(Amortized Analysis)。

摊还分析是一种分析操作序列平均代价的技术，由于难以衡量单次操作的最坏代价，所以我们从总代价的角度出发，使用总代价除以操作次数，得到平均每次操作花费代价。

具体而言，常见的分析方法有三种：

1. 聚合分析：直接计算总代价，再除以操作次数；
2. 记账法；
3. 势能法。

本文中使用的主要是第一种方法。

此外，本文也使用了记账法，记账法的原理是分配 credit 并记录每次消耗，后文会详细阐述。

3.2 传统工作

首先介绍论文中提到的相关工作。

朴素方法 很自然想到，最直接实现可变长数组的方案就是维护一个大小恰好等于当前元素数 N 的固定数组，每次 grow 或 shrink 时都触发全量重建。它的存储空间显然是 $N + O(1)$ ，可以达到理论最优，但是 grow 或 shrink 的摊还代价为 $\Theta(N)$ ，并且临时空间最大为 $2N + O(1)$ 。

Geometric Expansion 第二种传统方法就是前文提到过的几何增长策略，用空间换取时间，通过预分配额外容量降低重建成本。它的具体方法如下：

假设增长因子为 $1+\alpha$ ，初始分配大小为 C 的数组。当 N 个元素填满后，分配大小为 $(1+\alpha)N$ 的新数组并复制全部元素；当元素数降至 $\frac{N}{1+\alpha}$ 以下时，再缩小数组。使用摊还分析可知，该数据结构的存储空间为 $O(N)$ ，grow 的均摊代价为 $N/(\alpha N) = 1/\alpha$ 。

可以看到，如果我们增大 α ，虽然摊还代价小了，但是我们总的存储空间增加，因此，传统方法的局限就在于此，难以平衡空间和时间。根本原因在于粒度太粗，也就是现在的空间划分只是计算的平均空间，后续作者针对这种不足提出了本文的改进。

3.3 理论脉络与下界背景

前面的几种方法可以看成一条逐步省空间的路线。几何扩容用线性级别的空闲容量换来 $O(1)$ 均摊更新时间[2]；Sitarski 的 hashed array tree 和 Brodnik 等人的结构继续把额外空间降到 $O(\sqrt{N})$ [3, 4]。Tarjan 和 Zwick 的工作正是从这个 $\sqrt{N}$ 出发，但他们关心的不是再改一个增长因子，而是重新检查这个下界究竟由哪些空间组成[1]。

3.3.1 Brodnik 等人的空间下界

Brodnik 等人的结果说明，如果只看整个运行过程中出现过的最大空间，那么某个时刻一定会用到 $N + \Omega(\sqrt{N})$ 空间[4]。它的直觉可以概括成两种情况：块太多，需要保存大量指针；块太少，又必然存在一个很大的块，而申请这个大块时会出现临时空间峰值。两种情况都会带来 $\Omega(\sqrt{N})$ 的额外空间。这里先把它作为后续构造的背景，完整的分类证明放在下一章介绍核心方法时展开。

3.3.2 Tarjan–Zwick 对下界视角的扩展

Tarjan 和 Zwick 并不是否定这个下界，而是注意到上面的两种情况对应两类不同的空间：指针属于平时一直保留的额外空间，大块则主要在扩缩容过程中短暂出现。把二者分别记为 $s(N)$ 和 $t(N)$ 后，原来的讨论可以推广为

$$s(N)t(N) = \Omega(N).$$

这条式子的意思很直接：平时留下的空间越少，扩缩容时就越可能需要申请更大的块。比如取 $s(N) = N^{1/r}$ ，就必须有 $t(N) = \Omega(N^{1-1/r})$ 。到这里，相关工作只给出了构造应该达到的目标；下一章将先完整说明这个下界，再介绍怎样用分层结构达到它。

为避免章节编号混淆，下面提到的“第 6 节”“第 7 节”均指 Tarjan 和 Zwick 原论文中的章节；本文自身的章节会明确写成“本文第几章”。

4 论文核心方法

4.1 整体构造与技术路线

4.1.1 整体构造与技术路线：从空间下界到构造目标

在进入本文的具体数据结构构造之前，作者首先从已有的下界出发，重新审视 resizable array 问题中的空间开销。直观上，Brodnik 等人的数据结构已经给出了 $N + O(\sqrt{N})$ 大小的空间使用，这暗示在平时存储空间和扩容过程中的空间开销之间， $\sqrt{N}$ 可能是某种最优平衡。本文的关键观察是：这个下界中混合了两类不同的空间。

一种是数据结构在稳定状态下为了存储并访问数组所需要的空间；另一种是 grow 或 shrink 过程中短暂出现的临时空间。作者正是通过区分这两类空间，重新打开了 $N + O(\sqrt{N})$ 这一传统界限。

Theorem 4.1. Brodnik 等人的下界可以表述为：任何维护 resizable array 的数据结构，在某些时刻都必须使用 $N + \Omega(\sqrt{N})$ 空间。这个结论即使只考虑 grow 和 access 操作也成立[4, 1]。

其证明思路可以理解为对 block 数量进行分类讨论。假设经过 N 次 grow 操作后，数组中有 N 个元素，这些元素被存放在若干个动态分配的连续内存块中。设当前一共有 k 个内存块。

第一种情况是 $k \geq \sqrt{N}$ 。由于每一个内存块都必须能够被数据结构访问到，因此每个块至少需要一个指针指向它。这样，仅指针所需要的空间就至少为 k 。而原始数据本身至少需要 N 个空间，所以总空间至少为

$$N + k \geq N + \sqrt{N}.$$

这说明当 block 数量较多时，额外空间主要来自指针开销。也就是说，这一部分对应的是平时为了组织和访问这些 block 而长期存在的存储空间。

第二种情况是 $k < \sqrt{N}$ 。此时， N 个元素被放入少于 $\sqrt{N}$ 个 block 中。根据抽屉原理，必然存在一个 block 的大小大于 $\sqrt{N}$ 。设这个大 block 是在某一次 grow 操作中被申请出来的，当时数组长度为 $N' \leq N$ 。在该 block 刚刚申请出来的瞬间，它还没有真正承载原来的元素，而原来的 N' 个元素仍然存放在旧的 block 中。因此，在这一瞬间，除了旧的 N' 个元素空间之外，还额外出现了一个大小超过 $\sqrt{N'}$ 的新 block，于是当时总空间至少为

$$N' + \Omega(\sqrt{N'}).$$

这说明当 block 数量较少时，必然存在某个很大的 block，而这个大 block 在申请时会产生显著的临时空间。

因此, Theorem 4.1 的两个 case 实际上分别对应两类空间来源: case 1 说明的是平时存储和访问所需的指针空间; case 2 说明的是某一时刻突然申请一个大 block 时产生的临时空间。

这也是本文区分平时空间和额外临时空间的原因。

Theorem 4.2. 在 Theorem 4.1 的基础上, 作者进一步给出了更一般的乘积下界。对于任意 $(s(N), t(N))$ -implementation, 作者证明了如下乘积下界[1]:

$$s(N)t(N) \geq N.$$

其中, $s(N)$ 表示平时存储数组时允许的额外空间, $t(N)$ 表示 grow 或 shrink 过程中允许的临时额外空间。

这个证明与 Theorem 4.1 的思路保持一致, 仍然可以从抽屉原理出发。若数据结构在平时只允许 $N + s(N)$ 的空间, 那么除了 N 个真实元素之外, 最多只能有 $s(N)$ 量级的额外空间用于指针、索引或 block 管理信息。特别地, 由于每个 block 至少需要被某个指针或索引结构访问到, 我们可以认为当前 block 的数量至多为 $s(N)$ 量级。

于是, N 个元素分布在至多 $s(N)$ 个 block 中, 必然存在一个 block 的大小不小于

$$\frac{N}{s(N)}.$$

当这个 block 被申请出来时, 它尚未装入旧元素, 而旧元素仍然存放在原来的 block 中。因此, 在这一瞬间, resize 过程至少需要额外容纳这样一个大小为 $N/s(N)$ 量级的 block。于是有

$$t(N) \geq \frac{N}{s(N)}.$$

进一步得到

$$s(N)t(N) \geq N.$$

这个结论可以看成 Theorem 4.1 的加强版本。因为在 Theorem 4.1 中, 若取 $s(N) = t(N) = \sqrt{N}$, 就正好得到原来的 $N + \Omega(\sqrt{N})$ 下界。

更一般地, 如果我们希望平时的额外存储空间达到

$$s(N) = O(N^{1/r}),$$

那么由乘积下界可知, resize 过程中就不可避免地需要

$$t(N) \geq \Omega(N^{1-1/r}).$$

这说明, 如果我们想把稳定状态下的空间压缩到 $N + O(N^{1/r})$, 就必须接受 grow 或 shrink 过程中短暂使用 $N + O(N^{1-1/r})$ 的临时空间。

由下界引出的构造目标。 以上两个定理给出了本文整体构造的理论目标：作者希望围绕乘积下界 $s(N)t(N) \geq N$ 给出相匹配的数据结构。换言之，既然下界已经说明

$$s(N) = O(N^{1/r}) \implies t(N) = \Omega(N^{1-1/r}),$$

那么接下来的目标就是构造一个数据结构，使其真正达到

$$s(N) = O(N^{1/r}), \quad t(N) = O(N^{1-1/r}).$$

因此，本文后续构造部分的基本路线可以概括为：先通过下界区分平时空间和临时空间，然后围绕这个下界给出相匹配的结构。构造方面，最基本的 $r = 2$ 情况已经由 Brodnik 等人的 $N + O(\sqrt{N})$ 结构给出；随后作者给出特殊的 $r = 3$ 构造，再进一步推广到一般的 r 。

总结一下：这部分先说明 $s(N)t(N) \geq N$ 的理论最优，再通过 $r = 3$ 的例子展示如何突破稳定状态下的 $N + O(\sqrt{N})$ 空间，最后给出一般 r 的多层 block 构造。

4.2 空间优化机制

本节进一步解释上述分层构造中的空间优化机制。核心思想不是消除扩缩容时的数据搬移，而是把空间开销区分为稳定状态下的额外空间和扩缩容过程中的临时空间，并有意识地把较大的空间需求限制在短暂的块合并、拆分和重建过程中[1]。

4.2.1 $r = 3$ 的两级块结构

论文首先讨论一个 $(O(N^{1/3}), O(N^{2/3}))$ -implementation[1]。令参数 B 满足

$$N^{1/3} \leq B \leq 4N^{1/3}.$$

数据结构使用大小为 B^2 的大块存储数组中已经稳定下来的部分，并使用至多 $2B$ 个大小为 B 的小块保存数组末端。所有大块始终保持填满；小块中至多有一个部分填充的块，并可以额外保留一个空块。两类数据块分别由两个索引块管理，其布局与合并关系如图 2 所示。

这一设计可以看作对 hashed array tree 的再次分层。如果直接使用大小约为 $N^{2/3}$ 的数据块，最后一个数据块可能只存有少量元素，稳定状态下会浪费 $O(N^{2/3})$ 空间。现在改用大小约为 $N^{1/3}$ 的小块表示末端尚未填满的部分，尾部浪费和索引开销都只需 $O(N^{1/3})$ 空间。

这种压缩并非没有代价。当 $2B$ 个小块均已填满时，结构会将其中 B 个小块的元素复制到一个新申请的、大小为 B^2 的大块中；当小块耗尽而数组继续执行 **Shrink** 时，则需要把一个大块拆成 B 个小块。因此，合并或拆分过程中会短暂使用

$$O(B^2) = O(N^{2/3})$$

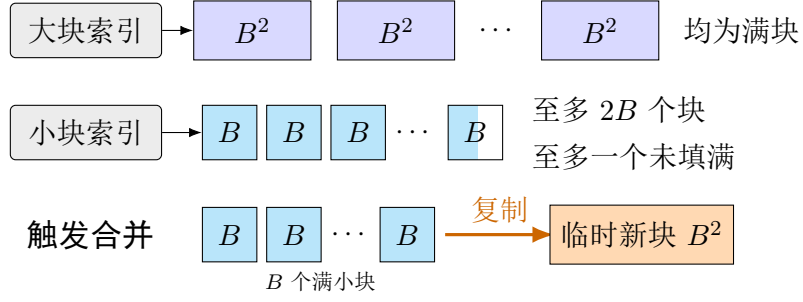


图 2: 参数 $r = 3$ 时的大块、小块与合并过程（根据文献 [1] 重绘）。

的额外空间。由此得到的正是“较小稳定空间、较大临时空间”的折中：平时只保留 $O(N^{1/3})$ 额外空间，而扩缩容过程允许 $O(N^{2/3})$ 的临时峰值。

4.2.2 一般 r 的分层块结构

对于任意整数 $r \geq 2$ ，第 6 节把两级结构推广为 $r - 1$ 个层级[1]。结构维护参数 B ，使得

$$N^{1/r} \leq B < 4N^{1/r},$$

并使用大小为

$$B, B^2, \dots, B^{r-1}$$

的数据块。大小超过 B 的块始终填满；最低层大小为 B 的块中，至多最后一个没有填满。每种块大小对应一个索引块，因此一共维护 $r - 1$ 个索引块。

各层块的数量可以理解为 N 的一种冗余 base- B 表示。若只支持 **Grow**，每层保留至多 B 个块即可：当某层积累了 B 个大小为 B^i 的满块时，将它们合并成一个大小为 B^{i+1} 的块。为了同时支持 **Shrink**，论文把每层的容量放宽到至多 $2B$ 个块。当最低层没有块可供删除时，结构找到最近的非空高层，并将一个高层块逐级拆成更小的块。这个冗余区间使合并与拆分之间留有缓冲，避免连续操作反复触发相反方向的重构。

稳定状态下，数据块除最低层的最后一个块外均为满块，主要额外开销来自各层索引、计数器和少量未使用位置。每层索引规模为 $O(B)$ ，共有 $r - 1$ 层，故稳定额外空间为

$$O(rB) = O(rN^{1/r}).$$

一次合并所申请的最大新块大小为 B^{r-1} ，因此扩缩容过程中的额外空间为

$$O(B^{r-1}) = O(N^{1-1/r}).$$

第 6 节由此得到

$$(O(rN^{1/r}), O(N^{1-1/r}))\text{-implementation.}$$

4.2.3 访问时间与均摊更新时间

分层并不意味着按下标访问必须逐层查找。论文令 B 为 2 的幂，并把各层块数压缩为机器字中的冗余 base- B 计数器。借助 **msb**、**lsb**、位掩码和移位操作，可以在 word-RAM 模型下以 $O(1)$ 最坏情况时间确定目标元素所在的层、数据块及块内偏移。因此，**Get** 和 **Set** 仍保持常数时间[1]。

合并与拆分虽然可能移动整个块，但其代价可以分摊到一段操作序列上。每个新元素先进入最低层，随后至多沿 $r - 1$ 个层级向上移动；拆分的代价也可以由两次同级拆分之间积累的 **Shrink** 操作支付。因此，**Grow** 和 **Shrink** 的均摊代价为 $O(r)$ 。本章只保留这一结构直觉，完整的 credit 分析和相应下界将在后续理论分析中讨论。

4.2.4 去除稳定空间中的 r 因子

当 r 是常数时， $O(rN^{1/r})$ 与 $O(N^{1/r})$ 在渐近意义下没有区别；但当 $r = r(N)$ 随 N 增长时，第 6 节结果中的 r 因子不能忽略。第 7 节为此提出数据结构转换。它针对一类标准实现：临时空间峰值主要出现在申请新块，并按顺序把若干旧块内容复制到新块的过程中。

设一个标准结构的参数原本为

$$(s(N), O(N)),$$

并给定非递减阈值函数 $t(N)$ 。当它试图申请长度超过 $t(N)$ 的新块时，不再申请单个完整大块，而是顺序申请若干个大小至多为 $t(N)$ 的小块，并保存指向这些小块的额外指针。这样，任一时刻新申请而尚未填满的空间被控制在 $O(t(N))$ ；另一方面，拆分大块所需的额外指针数为 $O(N/t(N))$ 。引理 7.2 给出的转换结果为[1]

$$\left(s(N) + O\left(\frac{N}{t(N)}\right), s(N) + O(t(N)) \right)\text{-implementation,}$$

同时不增加访问操作的渐近最坏情况时间，也不增加扩缩容操作的渐近均摊时间。

为了得到最终界，论文取

$$t(N) = N^{1-1/r},$$

并把该转换应用于第 6 节中参数为 $2r$ 的标准结构。输入结构的稳定额外空间为

$$O(rN^{1/(2r)}).$$

定理 7.3[1] 所处理的参数范围为

$$r \leq \frac{1}{2} \frac{\log N}{\log \log N}$$

在此范围内，上述空间项以及新增的

$$O(N/t(N)) = O(N^{1/r})$$

指针空间都可被 $O(N^{1/r})$ 吸收。最终得到

$$(O(N^{1/r}), O(N^{1-1/r}))\text{-implementation,}$$

并继续支持 $O(1)$ 最坏情况访问和 $O(r)$ 均摊扩缩容时间。这个转换说明，论文的最终空间界并非只来自增加层数，而是来自“分层表示”与“限制单次物理分配规模”两步优化的结合。

5 正确性证明与理论分析

5.1 上界分析

5.1.1 上界分析：一般 r 构造的均摊代价

对于一般 r 的构造，

$$(O(rN^{1/r}), O(N^{1-1/r}))\text{-implementation,}$$

需要证明 grow 和 shrink 的均摊成本均为 $O(r)$ 。

这里的一般构造指的是（实际上上文已经阐明，这里简单提及）：令 $B \approx N^{1/r}$ ，数据被存放在大小为

$$B, B^2, \dots, B^{r-1}$$

的多层 blocks 中。每一层最多维护 $2B$ 个 blocks。于是，从空间角度看，每一层的 index block 大小为 $O(B)$ ，一共有 $r-1$ 层，因此稳定状态下的额外存储空间为

$$O(rB) = O(rN^{1/r}).$$

另一方面，resize 过程中最大的临时申请发生在分配一个大小为 B^{r-1} 的 block 时，因此临时空间为

$$O(B^{r-1}) = O(N^{1-1/r}).$$

所以该结构首先满足

$$(O(rN^{1/r}), O(N^{1-1/r}))\text{-implementation}$$

的空间要求。

下面主要说明 grow/shrink 的均摊代价。前面的构造其实已经隐含了这个证明思想：因为最多有 $r - 1$ 层，所以对于任意一个元素，它最坏也只会在这几层之间被移动 $O(r)$ 次。换句话说，从直观上看，总的元素移动代价至多为 $O(Nr)$ ，而因为共有 N 个元素，均摊到每个元素上就是 $O(r)$ 。这是一个直觉证明，更加严谨的证明使用了记账法。

Grow 的均摊代价。 论文将一次 Grow 或 Shrink 的成本定义为 item assignments 的次数。对于 Grow 来说，普通插入一个新元素的成本为 1；如果触发 Combine-Blocks，则还要计算合并过程中元素复制的成本。

作者为 level i 中的每个元素分配

$$r - i$$

个 credit。这里 level i 指的是元素当前位于大小为 B^i 的 block 中。新插入的元素总是先进入 level 1，因此初始获得

$$r - 1$$

个 credit。当某个元素因为 Combine-Blocks 从 level i 被复制到 level $i + 1$ 时，它的 credit 从

$$r - i$$

变为

$$r - (i + 1),$$

正好减少 1，而这个减少的 credit 就用来支付这一次元素复制的代价。

因此，每个元素从 level 1 一路移动到更高层的过程中，credit 始终非负，且每一次复制都由 credit 支付。新插入元素时需要额外分配 $r - 1$ 个 credit，再加上插入本身的常数成本，所以 Grow 的均摊成本为

$$O(r).$$

这也对应了论文中“Combine-Blocks 的摊还成本为 0”的结论。

Shrink 的均摊代价。 Shrink 的分析比 Grow 更细一些，因为 Shrink 并不是简单地反向使用上面的元素 credit。我们先回忆一下 shrink 的操作。Shrink 始终删除数组末尾元素，也就是最新一段中最后的元素。如果最后一个 size- B block 中还有元素，那么直接删除即可，不需要复制元素，因此 item assignment 成本为 0。但是如果当前没有 size- B block，也就是 $n_1 = 0$ ，就必须调用 Split-Blocks，从更高层拆出一些低层 blocks，然后再删除末尾元素。

因此，Shrink 分为两种情况：

1. 最后一个 size- B block 中还有元素，直接删除，代价为 0；
2. 没有 size- B block，需要进行一次 Split-Blocks。

设这次 Split-Blocks 发生在 level k ，也就是当前最小的非空层为 k 。此时算法会将最后一个大小为 B^k 的 block 拆成更低层的 blocks。拆分后，levels 1 到 $k-1$ 中一共正好包含 B^k 个元素：对于 level $k-1, k-2, \dots, 2$ ，每一层各生成 $B-1$ 个对应大小的 blocks，也就是 $B-1$ 个 size- B^{k-1} blocks、 $B-1$ 个 size- B^{k-2} blocks，依次直到 $B-1$ 个 size- B^2 blocks；最后在 level 1 生成 B 个 size- B blocks。也就是说，一次 split 大致可以理解为把一个 B^k 级别的大块向下拆成若干层小块，从而重新暴露出可以继续 shrink 的 size- B blocks。

关键观察是：两次同级别的 level k split 之间，至少要经历 B^k 次 Shrink。原因是，一次 level k split 之后，低层 $1, \dots, k-1$ 中已经有 B^k 个元素；如果下一次还想再次发生 level k split，就必须先把这些低层元素全部删掉。因此，中间至少发生 B^k 次 Shrink。论文还指出，如果前一次不是同级 split，而是一次 Combine-Blocks 创建了一个新的 size- B^k block，那么在该 combine 之后，levels 1 到 $k-1$ 中也至少有 B^k 个元素，所以到下一次 level k split 之间同样至少有 B^k 次 Shrink。

为了支付 Split-Blocks 的成本，作者给每一次 Shrink 操作额外分配 level credit：每次 Shrink 都给每一层增加 3 个 credit。因此，如果一次 level k split 发生，那么在这之前 level k 至少已经积累了

$$3B^k$$

个 credit。

以上是 Shrink 获得的 credit，现在分析需要支付的成本。成本包括两部分。

第一部分是实际复制成本。拆开一个 size- B^k block，需要复制其中的 B^k 个元素，因此实际 item assignment 成本为

$$B^k.$$

第二部分是给移动后的元素补充 credit。因为 Split-Blocks 会把原来在 level k 的元素移动到更低层，而低层元素未来可能还会被向高层复制，所以需要补充元素 credit。若一个元素从 level k 被移动到 level i ，其中 $i < k$ ，那么它需要额外补充的 credit 数量为

$$k - i.$$

根据 split 后各层元素的分布，需要补充的 credit 总量为

$$B(k-1) + (B-1) \sum_{i=1}^{k-1} (k-i)B^i.$$

其中第一项和求和式共同对应 level 1 中的 B^2 个元素，其余项对应 level 2 到 level $k-1$ 中的元素。

接下来对该式进行上界估计：

$$B(k-1) + (B-1) \sum_{i=1}^{k-1} (k-i)B^i \leq B \sum_{i=1}^{k-1} (k-i)B^i.$$

令 $j = k - i$ ，则有

$$B \sum_{i=1}^{k-1} (k-i)B^i = B \sum_{j=1}^{k-1} jB^{k-j} \leq B^{k+1} \sum_{j \geq 1} jB^{-j}.$$

利用无穷级数

$$\sum_{j \geq 1} jB^{-j} = \frac{B}{(B-1)^2},$$

可得

$$B^{k+1} \sum_{j \geq 1} jB^{-j} = \frac{B^{k+2}}{(B-1)^2} \leq 2B^k,$$

其中最后一步使用了 $B \geq 4$ 。

因此，一次 level k 的 Split-Blocks 所需支付的总成本不超过

$$B^k + 2B^k = 3B^k.$$

而在该 split 发生之前，level k 已经至少积累了 $3B^k$ 个 credit，所以这些 credit 足以支付本次 split 的实际复制成本以及移动元素后的 credit 补充。因此，Split-Blocks 的摊还成本为 0。

Rebuild 的成本。 最后还需要考虑 Rebuild。Grow 或 Shrink 都可能导致参数 B 不再适合当前的 N ，因此算法会在 $N = B^r$ 时将 B 翻倍，或在 $N = (B/4)^r$ 时将 B 减半并重建结构。论文中同样使用 credit 支付 Rebuild 成本：每次 Grow 或 Shrink 再额外给整个数据结构增加 $2r$ 个 credit。这样虽然会把每次操作的均摊成本再增加 $O(r)$ ，但仍然保持在 $O(r)$ 范围内。

当 Rebuild 发生时，两次 Rebuild 之间已经经过了线性数量的 Grow 或 Shrink 操作，因此数据结构已经积累了足够多的 credit。Rebuild 的实际成本为 N ，而重建后为所有元素重新分配 credit 的总量至多为 $(r-1)N$ ，因此积累的全局 credit 足以支付 Rebuild 的成本和重新分配 credit 的成本。所以 Rebuild 的摊还成本也可以视为 0。

所以，Grow 和 Shrink 的均摊成本为 $O(r)$ （相当于主动增加 $O(r)$ 个 credit），其他重构操作的成本都可以由这些 credit 支付。

5.1.2 访问与修改操作的最坏 $O(1)$ 时间

对于访问与修改操作的最坏 $O(1)$ 时间，首先有一个直接的朴素思路：从高层到低层扫描，判断待访问元素落在哪一层。由于一般 r 的构造中最多有 $r-1$ 层，这样做的

复杂度为 $O(r)$ 。这一方法虽然直观，但还不能达到论文所要求的 worst-case $O(1)$ 。因此，作者进一步利用了广义 B 进制表示，将“寻找元素所在层”这一过程压缩到常数时间。

具体来说，令 n_0 表示最后一个部分填充的 size- B block 中的元素个数；令 n_1 表示 full size- B blocks 的数量；更一般地， n_j 表示 full size- B^j blocks 的数量。由于每一层的 block 数量都不超过 $2B$ ，因此有 $n_j \leq 2B$ 。于是当前数组长度可以写成一种冗余的 base- B 表示：

$$N = \sum_{j=0}^{r-1} n_j B^j.$$

这里之所以说是冗余表示，是因为普通 base- B 表示中每一位应小于 B ，而这里每个 n_j 可以达到 $2B$ 。不过这并不会破坏访问过程，反而正是 redundant base- B counter 的体现。

接下来定义

$$N_k = \sum_{j=0}^{k-1} n_j B^j.$$

直观上， N_k 表示低于 level k 的那些 blocks 中一共存放了多少元素。由于本文的数据结构中较高层 blocks 存放数组前面的、较稳定的元素，而较低层 blocks 存放靠近末尾的新元素，因此判断一个元素在哪一层，可以等价地从数组末尾往前数。

设要访问的元素下标为 i ，其中 $0 \leq i < N$ 。令

$$x = (N - 1) - i.$$

这里 x 表示该元素从数组末尾倒数的位置。于是，第 i 个元素位于 level k 当且仅当

$$N_k \leq x < N_{k+1}.$$

这个式子是访问证明中的核心。它说明，一个元素在哪一层，并不是由前面有多少元素决定，而可以由它后面有多少元素决定；而从后往前看时，低层 blocks 正好排在前面，所以判断会更自然。

一旦找到了这个 k ，定位元素就非常容易。因为 level k 的 block 大小为 B^k ，所以该元素在 level k 中的相对偏移为

$$x - N_k.$$

于是它所在的 block 编号和 block 内偏移分别可以由除法和取模得到。由于论文中取 $B = 2^b$ ，所以 B^k 也是二的幂，除法和取模都可以用 shift 和 mask 在常数时间内完成。因此，真正困难的地方只剩下一个：如何在 $O(1)$ 时间内找到满足 $N_k \leq x < N_{k+1}$ 的层数 k 。

为此，作者首先将 x 写成标准 base- B 表示，并令 ℓ 为其最高非零位。由于 $B = 2^b$ ，这个最高非零位可以通过机器字上的最高有效位操作得到：

$$\ell = \left\lfloor \frac{\text{msb}(x)}{b} \right\rfloor.$$

这里 $\text{msb}(x)$ 表示 x 的最高有效 bit 的位置。也就是说， ℓ 大致刻画了 x 的数量级。

接下来需要说明，找到 ℓ 后，真正的层数 k 不会离 ℓ 太远。论文利用每个 $n_j \leq 2B$ 这一性质，得到

$$N_{\ell-1} = \sum_{j=0}^{\ell-2} n_j B^j \leq 2B \sum_{j=0}^{\ell-2} B^j \leq 3B^{\ell-1} \leq B^\ell \leq x.$$

因此有

$$x \geq N_{\ell-1},$$

从而可以推出

$$k \geq \ell - 1.$$

这一步非常关键。它说明我们不需要从所有层开始找，而是只需要从 $\ell - 1$ 附近开始判断。

此时可以分三种情况讨论。第一，如果

$$x < N_\ell,$$

那么结合 $N_{\ell-1} \leq x$ ，可知

$$N_{\ell-1} \leq x < N_\ell,$$

因此

$$k = \ell - 1.$$

第二，如果

$$N_\ell \leq x < N_{\ell+1},$$

那么直接得到

$$k = \ell.$$

第三，如果

$$N_{\ell+1} \leq x,$$

说明 x 已经越过了 level ℓ 对应的范围。此时需要继续向更高层寻找下一个非空层。也就是说， $k = \ell'$ ，其中 $\ell' > \ell$ ，并且 ℓ' 是满足 $n_{\ell'} > 0$ 的最小下标。

最后的问题就是：如何在 $O(1)$ 时间内找到这个下一个非空层。由于每个 n_j 可能达到 $2B$ ，论文将它拆成两部分：

$$n_j = n_j^0 + Bn_j^1,$$

其中

$$0 \leq n_j^0 < B, \quad 0 \leq n_j^1 \leq 2.$$

然后将所有低位部分打包成一个机器字：

$$N_0 = (n_{r-1}^0, \dots, n_1^0, n_0^0)_B,$$

再将所有高位部分打包成另一个机器字：

$$N_1 = (n_{r-1}^1, \dots, n_1^1, n_0^1)_B.$$

这样，某一层 j 非空当且仅当 $n_j^0 > 0$ 或 $n_j^1 > 0$ 。因此，只要观察 $N_0 \vee N_1$ 中对应 digit 是否非零，就可以判断某一层是否非空。

为了找到 ℓ 以上的第一个非空层，只需要用 mask 去掉低于等于 ℓ 的部分，然后找剩余部分中的最低有效位即可。由于最低有效位 `lsb` 也可以通过机器指令或由 `msb` 间接得到，这一步也可以在常数时间内完成。由此，作者成功在 $O(1)$ 时间内找到了目标元素所在的 level k 。

得到 k 后，再根据 N_k 计算层内偏移，并用 `shift/mask` 得到 block 编号和 block 内位置，就可以直接访问该元素。修改操作与访问操作完全相同，只是在定位到对应位置后将该位置的值替换掉。因此，`access` 和 `modify` 都可以支持 worst-case $O(1)$ 时间。

总的来说，这一部分的核心思想是：朴素扫描层级需要 $O(r)$ 时间，而论文通过把各层 block 数量编码成冗余 base- B 表示，再利用机器字上的 `msb`、`mask`、`shift` 等操作，将寻找元素所在层压缩为常数时间。换句话说，访问与修改之所以能做到 $O(1)$ ，关键不在于多层结构本身简单，而在于作者把层定位问题转化成了一个机器字上的位运算问题。

5.2 下界分析

本文第 3 章已经说明，若稳定状态下的额外空间为 $s(N)$ ，扩扩容过程中允许的临时额外空间为 $t(N)$ ，则二者满足

$$s(N)t(N) = \Omega(N).$$

这一结论刻画的是空间之间的权衡。本节转而讨论更新时间：当稳定额外空间被压缩到 $O(rN^{1/r})$ 时，`Grow` 的均摊代价是否还可能低于 $O(r)$ 。论文第 8、9 节通过 `growth game` 证明，在一类标准实现中答案是否定的[1]。

5.2.1 标准实现及下界的适用范围

论文把块迁移遵循特定形式的结构称为标准实现。每当结构申请一个新块 B 时，它立即将若干旧块 $A_1, A_2, \dots, A_q$ 中的元素按顺序复制到 B ；一个旧块复制完毕后随即

释放。此次合并结束后，结构重新回到只占用 $N + s(N)$ 空间的稳定状态。因而，临时空间只在“申请新块并搬移旧块”的过程中短暂存在。

这一限制使真实的块合并能够被组合游戏刻画。论文前面给出的分层结构以及经典分块结构都属于此类实现，但该定义并不直接覆盖把一次迁移长期分散到许多操作、使用更复杂编码或不遵循上述复制方式的算法。因此，后面的时间下界必须连同标准实现这一前提一起理解。

5.2.2 (N, k, ℓ) -growth game

growth game 依次插入 N 个元素，并用 k 个初始为空的子数组 $A_1, A_2, \dots, A_k$ 保存它们。空子数组位于序列前部；第一个非空子数组至多保留 ℓ 个空位，其余非空子数组均为满块。若 a_i 表示 A_i 中的元素数，则一次插入分为三种情况：

1. 第一个非空子数组仍有空位，直接写入新元素，代价为 1；
2. 所有非空子数组都已填满但仍有空子数组，申请一个大小为 $\ell + 1$ 的新块，写入一个元素并留下 ℓ 个空位，代价为 1；
3. 所有 k 个子数组都非空且已填满，选择某个 $i \in [k]$ ，将 $A_1, \dots, A_i$ 与新元素合并成新的 A_i ，并令前 $i - 1$ 个子数组重新为空。其代价为

$$1 + \sum_{j=1}^i a_j,$$

即一次新元素写入加上所有被搬移旧元素的赋值次数。

图 3 展示了选择 $i = 2$ 时，新元素与前两个子数组合并的过程。

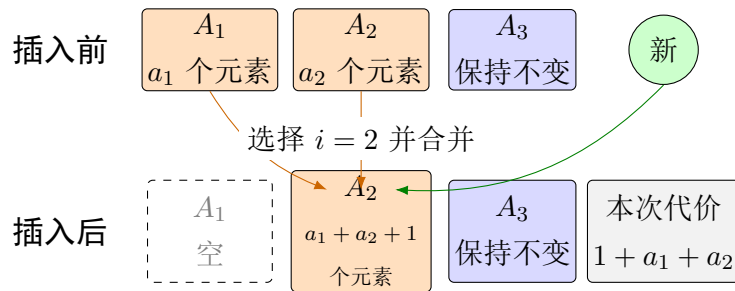


图 3: growth game 中选择 $i = 2$ 的一次合并示例（根据文献 [1] 整理）。

游戏忽略内存申请、释放和指针更新的成本，也不限制一次操作的临时空间，因此它给算法提供了比真实模型更宽松的条件。记完成游戏的最小总代价为 $C_{N,k,\ell}$ ，对应的最小均摊代价为

$$A_{N,k,\ell} = \frac{C_{N,k,\ell}}{N}.$$

5.2.3 二项式阈值与游戏下界

参数 ℓ 表示第一个非空子数组中最多可以留下多少个空位。它看起来会影响游戏，但实际上只相当于把若干次插入合并处理：一个新块建立后，第一次插入写入一个元素，接下来的 ℓ 次插入只是依次填满预留位置，并不会触发新的合并。因此，当 N 能被 $\ell + 1$ 整除时，可以把连续 $\ell + 1$ 次插入看成一次成组插入。引理 8.2 据此得到[1]

$$C_{N,k,\ell} = (\ell + 1)C_{N/(\ell+1),k,0}, \quad A_{N,k,\ell} = A_{N/(\ell+1),k,0}.$$

这个等式说明，预留空位并没有真正消除元素搬移，只是把原游戏压缩成一个规模更小的游戏。后面只需研究 $\ell = 0$ 的情形，并简记 $C_{N,k} = C_{N,k,0}$ 、 $A_{N,k} = A_{N,k,0}$ 。

接下来的问题是：随着插入次数增加，一个元素最少要经历多少次合并？定理 8.6 用二项式系数给出了精确答案[1]。若整数 $n \geq 0$ 满足

$$\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1,$$

则

$$C_{N,k} = (N+1)n - \binom{n+k}{k+1}.$$

这里不必复现定理 8.6 较长的归纳证明。对后面的时间下界来说，只需要理解参数 n 的作用：当 N 跨过相应的二项式阈值后，即使采用最优策略，也无法避免更多次元素搬移。推论 8.13 把这一点直接写成均摊代价的下界。若

$$\binom{n+k-1}{k} \leq N \leq \binom{n+k}{k} - 1,$$

则

$$A_{N,k} \geq \frac{k}{k+1}(n-1) \geq \frac{n-1}{2}.$$

右侧至少为 $(n-1)/2$ 。因此，如果能在真实数据结构对应的游戏中找到一个 $n = \Theta(r)$ ，并证明游戏规模已经越过它的二项式阈值，就能得到所需的 $\Omega(r)$ 均摊下界。下面的参数估计正是为了完成这一步。

5.2.4 从真实结构到 $\Omega(r)$ 下界

现在把游戏结论带回可变长数组。考虑一个标准实现，其稳定状态只使用

$$N + O(rN^{1/r})$$

空间。令 $m = rN^{1/r}$ ，并暂时忽略常数因子。为什么这里会出现 m ？因为数组在稳定状态下只有 $O(m)$ 的额外位置，这些位置要同时容纳块指针、管理信息和未填满的槽位，所以结构能够保留的块数和空位数也只能是 $O(m)$ 。

于是，可以把真实结构中的数据块对应为 growth game 中的子数组，把“申请一个新块并把若干旧块复制进去”对应为游戏中的一次合并。为避免下界依赖过多实现细节，论文使用的扩展游戏还允许算法任意选择需要合并的子数组。这比真实数据结构拥有更多自由，对算法更加有利。引理 8.17 和 8.18 证明，这些额外自由并不会降低游戏的最优总代价。因此，只要这个更宽松的游戏仍需付出 $\Omega(r)$ 均摊代价，真实的标准实现也不可能做得更好。

在这个对应关系中，块数和空位数都可以用 m 控制，所以真实结构可映射到扩展的 (N, m, m) -growth game。再利用引理 8.2 把预留空位成组压缩，并忽略不影响渐近结果的整除和取整问题，得到一个包含

$$N' = \Theta\left(\frac{N}{m}\right) = \Theta\left(\frac{1}{r}N^{1-1/r}\right)$$

次插入、 $k = m = rN^{1/r}$ 个子数组且 $\ell = 0$ 的游戏。

接下来要使用推论 8.13。正如上一小节所说，我们希望选择 $n = \Theta(r)$ ，再证明当前游戏规模 N' 已经超过对应的二项式阈值。论文取 $n = \beta r$ ，并令 $\beta = 1/2$ 。使用标准估计

$$\binom{x}{y} \leq \left(\frac{ex}{y}\right)^y$$

可得

$$\binom{\beta r + rN^{1/r}}{\beta r} \leq \left(\frac{e(\beta + 1)}{\beta}\right)^{\beta r} N^{\beta}.$$

左侧正是参数 $n = \beta r$ 所对应的二项式阈值。取 $\beta = 1/2$ 后，当 $r = O(\log N)$ 且 N 充分大时，证明使用下面的不等式把该阈值控制在 N' 以内：

$$r(3e)^{r/2}N^{1/2} \leq N^{3/4} \leq N^{1-1/r}.$$

也就是说，约化后的游戏规模足以越过某个 $n = \Theta(r)$ 的阈值。现在才能调用推论 8.13，得到

$$A_{N',k} \geq \frac{n-1}{2} = \Omega(r).$$

这就得到定理 9.1：任何稳定存储空间为 $N + O(rN^{1/r})$ 、且 $r = O(\log N)$ 的标准可变量数组，其 **Grow** 操作的均摊代价均为 $\Omega(r)[1]$ 。

原论文第 6、7 节的结构恰好达到 $O(r)$ 均摊扩缩容时间，所以上下界在标准实现范围内匹配。这里的限定不能省略：定理讨论的是标准实现和均摊代价，并没有证明所有可能的算法都必须满足同样的下界，也不能据此断言最坏情况时间一定无法改进。如何把结论推广到非标准算法，仍是论文留下的问题。

6 技术直觉与方法评价

6.1 技术直觉与示例解释

事实上，在这篇工作之前，没有区分过具体使用什么空间，最经典的 Brodnik 结论指出某些时刻会用到 $N + \Omega(\sqrt{N})$ 空间。

本文对这种空间进行细化，没有否认这种结论，而是指出了一个更一般性的规律，这是作者的深刻洞见。这是从特殊到一般的归纳思维。

所以这引出来了**第一个技术直觉**，就是旧下界中的 $\sqrt{N}$ 只是平时空间和临时空间相等的一个平衡点。问题的本质，作者在这里指出来，是一个乘积约束：

$s(N)t(N) \geq N$ 。当 $s(N) = t(N) = \sqrt{N}$ 时，正好回到 Brodnik 等人的结论；而如果希望平时存储空间进一步下降，比如 $s(N) = N^{1/r}$ ，那么临时 resize 空间就必须上升到 $t(N) = N^{1-1/r}$ 。这说明所谓最优空间并不是绝对的，而取决于我们如何定义和区分资源。

第二个技术直觉，是作者发现 resizable array 中的元素并不是完全同质的。数组只在末尾 grow 和 shrink，因此靠近末尾的元素是最新的、最容易被删除和改变的；而靠前的元素更老，也更稳定。本文的多层 block 构造正是利用了这种差异：新加入的元素先放在小 blocks 中，以便处理频繁的尾部变化；当这些元素逐渐变得稳定之后，再被合并进更大的 blocks 中，从而减少 block 数量和指针开销。这和之前的证明一样，就是中间证明的发现：新加入的元素都在低层或者同层后面，这正是作者发现了这一特性而设计出来的，从这个角度观察，之前的数据结构（多层更新）其实并不突兀，只是利用了 grow/shrink 只发生在末尾这一操作特性。

可以具体解释一下这个直觉是怎么转化为数据结构的：大小为 $B, B^2, \dots, B^{r-1}$ 的 blocks 对应不同的稳定层次。Level 1 的 size- B blocks 变化最频繁，类似计数器中的低位；更高层的 blocks 只有在低层积累到一定程度后才会改变，类似计数器中的高位。因此，base- B counter 在这里解释了为什么这种分层结构是自然的：低位频繁变化，所以用小块；高位很少变化，所以用大块。作者实际上是用计数器的进位结构刻画了元素稳定性的变化过程。

第三个技术直觉，是本文对冗余的理解很深。但在本文中，冗余是防止 grow 和 shrink 在边界附近反复震荡的缓冲机制。对于 shrink 来说，如果每一层刚好满 B 个 blocks 就合并，刚好不够就拆分，那么 grow/shrink 交替出现时，就可能刚合并完又立刻拆开，刚拆开又马上合并，导致大量无意义的数据搬移。本文使用 redundant base- B counter，让每一层最多允许 $2B$ 个 blocks。

具体为什么选择 $2B$ ？我们这里可以给出一个回答：若一层达到 B 个 blocks 就立即合并，则合并后该层可能被清空，随后 shrink 就可能迫使刚生成的高层 block 被重新拆开。相比之下，本文 redundant base- B counter：当某层达到 $2B$ 个 blocks 时，只

将其中 B 个合并到上一层，剩余 B 个 blocks 仍保留在当前层。因此，若一次合并生成了 size- B^k block，则低层仍至少保留 B^k 个元素；在下次同级 split 发生前，必须至少经过 B^k 次 shrink。这一缓冲保证了分裂所需的复制成本和 credit 补充这两个需求可以由此前的 shrink 操作摊还支付。

6.1.1 示例解释

数据结构的逐步变化如果全部放在正文中，会打断这里的讨论。更直观的图示和操作过程整理在附录文档 `notes/A_notes/直观演示.md` 中，可以结合前面的三点直觉阅读。

6.2 创新点、局限性与后续影响

前面已经介绍了数据结构和证明。本节换一个角度来看：这篇论文真正解决了什么，还有哪些结论不能说得太满[1]。

6.2.1 核心创新与理论贡献

论文的第一个贡献，是改变了讨论“空间最优”的方式。前面已经解释过 $s(N)t(N) = \Omega(N)$ 的含义，这里更值得强调的是它带来的视角变化： $\sqrt{N}$ 不再是唯一需要追求的数值，而只是稳定空间与临时空间相等时的一个平衡点。设计者可以根据使用场景选择其他位置，只是不能让两种空间同时任意变小[1]。

第二个贡献，是作者不只给出了下界，还真的构造出一族数组。对整数参数 r ，分层块结构和后面的数据结构转换可以实现

$$s(N) = O(N^{1/r}), \quad t(N) = O(N^{1-1/r}),$$

同时保持 $O(1)$ 最坏情况访问和 $O(r)$ 均摊扩缩容时间。可以把 r 理解成一个调节旋钮： r 越大，数组平时越紧凑，但结构的层数更深，更新时也要做更多工作。

第三个贡献，是用 growth game 分析块合并。这个游戏把具体的指针和块大小先放到一边，只保留“新元素放进哪个子数组、什么时候合并旧数组”这件事。它给出的 $\Omega(r)$ 均摊下界与构造的 $O(r)$ 上界相同。因此，在论文讨论的标准实现中，作者的结构不仅能做到这个复杂度，也不能再从渐近意义上做得更快。

6.2.2 模型假设与定理边界

不过，这里的“最优”有明确前提。首先， $O(1)$ 访问使用了 word-RAM 模型。原论文第 6.3 节把各层的块数放进少数机器字，再用 `msb`、`lsb`、位掩码和移位找到目标层，其中一种常数时间计算 `msb` 的方法还用到了乘法。这个做法在理论模型中没有问

题，但真实程序中的 r 往往很小，直接扫描几层或做一次二分可能反而更简单[1]。所以，理论上的常数时间不等于实际运行一定更快。

其次， r 也不能任意增大。最终的定理 7.3 要求

$$r \leq \frac{1}{2} \frac{\log N}{\log \log N}.$$

而且转换时输入的是原论文第 6 节中参数为 $2r$ 的标准结构[1]。因此，引用最终复杂度时不能把这些条件一起省略，否则会把论文的结论说得比实际更强。

最后，时间下界只证明到了标准实现（standard implementations）。这里的标准实现是指：申请新块后立即复制旧块，并在一次操作结束前重新满足稳定空间限制。作者认为，只要再对元素和指针加入合适的不可压缩性假设，下界应该也能推广到其他算法；但论文没有完成这部分证明[1]。所以目前只能说标准实现不能更快，不能直接说所有算法都不能更快。

6.2.3 均摊更新与工程可行性

论文给出的 Grow 和 Shrink 是 $O(r)$ 均摊时间，不是每一次操作都只花 $O(r)$ 时间。一个自然的问题是：能否把一次很大的复制拆到后面的多次操作中，也就是使用常见的后台重建？这里会遇到一个空间上的麻烦。

在最高层合并中，新块规模可达

$$\Theta(N^{1-1/r}).$$

如果复制在一次操作中完成，新块只算临时空间；如果复制被拆成许多步，新旧两份数据就会跨越多次操作同时存在，新块也就变成了平时要保留的空间。当 $r > 2$ 时，

$$N^{1-1/r} > N^{1/r},$$

也就是说，需要慢慢复制的新块可能比稳定空间预算还大。因此，普通的后台重建不能直接套到原论文第 6、7 节的结构上。不过，这只能说明常见方法不合适，并不能证明所有去摊还化方法都不存在。 $r = 2$ 时两种规模都是 $\Theta(\sqrt{N})$ ，这也解释了为什么 HAT 和 Brodnik 等经典结构可以得到最坏情况更新界[3, 4]。

从实现角度看，这个结构也明显比普通动态数组复杂。它要维护多层块目录、重建阈值、冗余 base- B 计数器和一些位操作，还会更频繁地经过指针访问不同内存块。一般程序更关心缓存局部性、分配器成本和代码复杂度，因此简单的几何扩容或浅层分块仍可能更合适。论文解决的是稳定额外空间受到严格限制时的理论问题，而不是在所有实际场景中替代 vector 或 ArrayList。

6.2.4 开放问题与后续意义

论文最后留下两个比较直接的问题[1]。第一个是把 $\Omega(r)$ 均摊下界从标准实现推广到所有算法，这需要更准确地描述元素和指针为什么不能被随意压缩。第二个是简化 growth game 的证明。现有证明给出了游戏的精确解，但篇幅很长；如果只需要 $\Omega(r)$ 这个渐近下界，也许存在更短的归纳或势能证明。

另一个很实际的问题是，小参数下的效果究竟怎样。例如，可以直接比较 $r = 2$ 、 $r = 3$ 的结构与 HAT、几何扩容数组，观察它们的缓存行为、分配次数和常数开销。即使论文的结构最终不直接用于工程，它仍然改变了提问方式：我们不再只问“哪个动态数组最省空间”，而是问愿意用多少临时空间和更新时间，来换取更小的长期空间。

7 总结

这篇论文从一个很简单的区别出发：数组平时多占的空间，和扩缩容那一刻临时多占的空间，不应该混在一起算。过去的 $N + \Theta(\sqrt{N})$ 结论并没有错，但 $\sqrt{N}$ 只是两种空间取得平衡时的结果。把它们分别写成 $s(N)$ 和 $t(N)$ 后，问题就从“能不能突破 $\sqrt{N}$ ”变成了“我们愿意怎样交换两种空间”[1]。

作者用参数 r 控制这种交换。元素先进入小块，积累到一定数量后再逐层合并到大块；这个过程很像 base- B 计数器的进位。最后得到的稳定空间是 $N + O(N^{1/r})$ ，扩缩容时的峰值空间是 $N + O(N^{1-1/r})$ 。访问仍然是 $O(1)$ 最坏情况时间，扩缩容是 $O(r)$ 均摊时间。 $r = 2$ 时回到熟悉的平方根级空间；继续增大 r ，平时会更省空间，但结构和更新都会更重。

两个下界说明了这种代价不是偶然的。空间方面，块太多会浪费指针，块太少又会在申请大块时产生峰值，因此有 $s(N)t(N) = \Omega(N)$ 。时间方面，growth game 说明标准实现若只保留 $N + O(rN^{1/r})$ 的稳定空间，Grow 的均摊代价就至少是 $\Omega(r)$ 。前者解释为什么两种空间不能一起变小，后者解释为什么省下长期空间以后必须多搬一些元素。

因此，论文真正有价值的地方不只是给出一个复杂的新数组，而是把“省空间”这件事说得更清楚：省的是平时空间还是临时空间，又要为此付出多少更新时间。当然，它仍是一项偏理论的工作。常数时间访问依赖 word-RAM 和位操作，更新时间只有均摊保证，时间下界也只覆盖标准实现。怎样把下界推广到一般算法，以及 $r = 2$ 、 $r = 3$ 的结构在真实程序中是否值得使用，仍然没有完整答案。

参考文献

- [1] Robert E. Tarjan and Uri Zwick. Optimal resizable arrays, 2023.

- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] Edward Sitarski. Algorithm alley: Hats: Hashed array trees. *Dr. Dobb's Journal of Software Tools*, 21(9):107, 1996.
- [4] Andrej Brodnik, Svante Carlsson, Erik D. Demaine, J. Ian Munro, and Robert Sedgwick. Resizable arrays in optimal time and space. In *Proceedings of the 6th International Workshop on Algorithms and Data Structures (WADS)*, pages 37–48, 1999.